

XXXXR0

views::cartesian_product

Draft Proposal, 2021-04-23

This version:

<http://wg21.link/PXXXX>

Author:

Sy Brand (sy.brand at microsoft dot com)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes `std::ranges::cartesian_product_view` for taking the cartesian product of multiple forward ranges.

Table of Contents

1	Motivation
2	Design
2.1	Minimum Range Requirements
2.2	Tuple or pair
2.3	reference_type
2.4	Empty cartesian product view
2.5	Common Range
2.6	Bidirectional Range
2.7	Random Access Range
2.8	Sized Range
2.9	Naming
2.10	Pipe Support
3	Implementation
4	Wording
4.1	Cartesian Product view [<code>range.cartesian</code>]
4.1.1	Overview [<code>range.cartesian.overview</code>]
4.1.2	Class template <code>cartesian_product_view</code> [<code>range.cartesian.view</code>]
4.1.3	Class template <code>cartesian_product_view::iterator</code> [<code>ranges.cartesian.iterator</code>]
4.1.4	Class template <code>cartesian_product_view::sentinel</code> [<code>ranges.cartesian.sentinel</code>]
5	Acknowledgements

§ 1. Motivation

[Cartesian product](#) is a fundamental mathematical construct. There should be a `cartesian_product_view` which generates the cartesian product of any number of ranges, e.g.:

Before	After
<pre> std::vector<int> a,b,c; for (auto&& ea : a) { for (auto&& eb : b) { for (auto&& ec : c) { use(ea,eb,ec); } } } </pre>	<pre> std::vector<int> a,b,c; for (auto&& [ea,eb,ec] : std::views::cartesian_product(a,b,c)) { use(ea,eb,ec); } </pre>

This is especially useful for composing with other views, or dealing with parameter packs of ranges:

Before	After
<pre> template <std::size_t N = 0, class F, class Res, class Tuple, class... Args> auto find_tuples_satisfying_impl(F f, Res& res, Tuple const& ranges, Args const&... args) requires(N == std::tuple_size_v<std::remove_cvref_t<Tuple>>) { if (std::invoke(f, args...)) { res.push_back(std::make_tuple(args...)); } } template <std::size_t N = 0, class F, class Res, class Tuple, class... Args> auto find_tuples_satisfying_impl(F f, Res& res, Tuple const& ranges, Args const&... args) { for (auto&& e : std::get<N>(ranges)) { find_tuples_satisfying_impl<N+1>(f, res, ranges, args..., e); } } template <class F, std::ranges::forward_range... Vs> requires(std::regular_invocable< F, std::ranges::range_reference_t<Vs>...>) auto find_tuples_satisfying(F f, Vs&&... vs) { std::vector<std::tuple<std::ranges::range_value_t<Vs>...>> res; find_tuples_satisfying_impl(f, res, std::tuple<std::forward<Vs&&>(vs)...>()); return res; } </pre>	<pre> template <class F, std::ranges::forward_range... Vs> requires(std::regular_invocable<F, std::ranges::range_reference_t<Vs>...>) auto find_tuples_satisfying(F f, Vs&&... vs) { return std::views::cartesian_product(std::views::filter([&f](auto&& e) { std::ranges::to<std::vector>(} </pre>

§ 2. Design

§ 2.1. Minimum Range Requirements

This paper requires all ranges passed to `cartesian_product_view` to be forward ranges. It would be possible to implement the type such that exactly one of the ranges could be an input range, but we don't consider this use-case strong enough for the implementation burden.

§ 2.2. Tuple or pair

A potential design would be to use `std::tuple` as the `value_type` and `reference_type` of `cartesian_product_view`. This paper uses `std::pair` if two ranges are passed and `std::tuple` otherwise. See [p2321](#) for motivation.

§ 2.3. reference_type

This paper uses `tuple-or-pair<ranges::reference_t<Vs>...>` as the reference type. See [p2214](#) for discussion of value types (particularly `pair` and `tuple`) as the `reference_type` for ranges, and [p2321](#) for wording of improvements for key use-cases.

§ 2.4. Empty cartesian product view

Trying to take the cartesian product view of 0 views will produce an `empty_view<tuple<>>`, in parity with Range-v3 and [p2321](#).

§ 2.5. Common Range

`cartesian_product_view` can be a common range either if all the underlying ranges are common, or if the underlying ranges are all sized and random access. This paper reflects this.

§ 2.6. Bidirectional Range

`cartesian_product_view` can be a bidirectional range if the underlying ranges are bidirectional and common, or if they are random access and sized. Non-common bidirectional ranges are not supported because decrementing when one of the iterators is at the beginning of its range would require advancing the iterator to end, which may be linear in complexity.

We don't consider non-common, random access, sized ranges as worth supporting, so this paper requires bidirectional and common.

§ 2.7. Random Access Range

`cartesian_product_view` can be a random access range if all the underlying ranges are random access and sized. Sized ranges are required because when the view is incremented, the new states of the iterators are calculated modulo the size of their views.

We can't think of too many use-cases for this and it adds a fair bit of implementation burden, but this paper supports the necessary operations.

§ 2.8. Sized Range

`cartesian_product_view` can be a sized range if all the underlying ranges are, in which case the size is the product of all underlying sizes. This is reflected in the paper.

§ 2.9. Naming

An alternative name is `std::ranges::product_view`. This paper uses `cartesian_product_view` as we believe it is more explicit in what its semantics are.

§ 2.10. Pipe Support

It may be possible to support syntax such as `vec1 | views::cartesian_product(vec2)` by either fixing the number of arguments allowed to the view, or adding a pipe operator to `cartesian_product_view` itself.

However, it's problematic for the same reason as `views::zip`, in that one cannot distinguish between the usages in `a | views::cartesian_product(b, c)` and `views::cartesian_product(b,c)` on its own. As such, this paper does not support this syntax.

§ 3. Implementation

There are implementations of a cartesian product view in [Range-v3](#), [cor3ntin::rangesnext](#), and [tl::ranges](#), among others.

§ 4. Wording

§ 4.1. Cartesian Product view [range.cartesian]

§ 4.1.1. Overview [range.cartesian.overview]

`cartesian_product_view` presents a `view` with a value type that represents the cartesian product of the adapted ranges.

The name `views::cartesian_product` denotes a customization point object. Given a pack of subexpressions `Es...`, the expression `views::cartesian_product(Es...)` is expression-equivalent to

- `*decay-copy*(views::empty<tuple<>>)` if `Es` is an empty pack,
- otherwise, `cartesian_product_view<views::all_t<decltype((Es))>...>(Es...)`.

[Example:

```
std::vector<int> v { 0, 1, 2 };
for (auto&& [a,b,c] : std::views::cartesian_product(v, v, v)) {
    std::cout << a << ' ' << b << ' ' << c << '\n';
    //0 0 0
    //0 0 1
    //0 0 2
    //0 1 0
    //0 1 1
    //...
}
```

-- end example]

§ 4.1.2. Class template `cartesian_product_view` [range.cartesian.view]

```
namespace std::ranges {
    template <class... Vs>
    concept cartesian-product-is-random-access = //exposition only
        ((random_access_range<Vs> && ...) &&
         (sized_range<Vs> && ...));

    template <class... Vs>
    concept cartesian-product-is-bidirectional = //exposition only
        ((bidirectional_range<Vs> && ...) && (common_range<Vs> && ...));

    template <class... Vs>
    concept cartesian-product-is-common = //exposition only
        (std::ranges::common_range<Vs> && ...)
        || cartesian-product-is-random-access<Vs...>;

    template<class... Ts>
    using tuple-or-pair = decltype([]{ // exposition only
        if constexpr (sizeof...(Ts) == 2) {
            return type_identity<pair<Ts...>>{};
        }
        else {
            return type_identity<tuple<Ts...>>{};
        }
    })::type;

    template<class F, class Tuple>
```

```

constexpr auto tuple-transform(F&& f, Tuple&& tuple) // exposition only
{
    return apply([&<class... Ts>(Ts&&... elements){
        return tuple-or-pair<invoke_result_t<F&, Ts>...>(
            invoke(f, std::forward<Ts>(elements))...
        );
    }, std::forward<Tuple>(tuple));
}

template<class F, class Tuple>
constexpr void tuple-for-each(F&& f, Tuple&& tuple) // exposition only
{
    apply([&<class... Ts>(Ts&&... elements){
        (invoke(f, std::forward<Ts>(elements)), ...);
    }, std::forward<Tuple>(tuple));
}

template<forward_range... Vs>
requires (view<Vs> && ...)
class cartesian_product_view
: public view_interface<cartesian_product_view<Vs...>> {
private:
    std::tuple<Vs...> bases_; //exposition only

    template<bool Const>
    struct iterator; //exposition only
    template<bool Const>
    struct sentinel; //exposition only
public:
    constexpr cartesian_product_view() = default;
    constexpr cartesian_product_view(Vs... bases);

    constexpr iterator<false> begin() requires (!simple_view<Vs> || ...);
    constexpr iterator<true> begin() const requires (range<const Vs> && ...);

    constexpr iterator<false> end()
    requires ((!simple_view<Vs> || ...) && cartesian-product-is-common<Vs...>);
    constexpr iterator<true> end() const
    requires (cartesian-product-is-common<const Vs...>);
    constexpr default_sentinel_t end() const
    requires (!cartesian-product-is-common<const Vs...>);

    constexpr auto size() requires (sized_range<Vs> && ...);
    constexpr auto size() const requires (sized_range<const Vs> && ...);
};

template <class... Vs>
cartesian_product_view(Vs&&...)->cartesian_product_view<all_t<Vs>...>;

namespace views { inline constexpr unspecified cartesian_product = unspecified; }
}

```

```
constexpr cartesian_product_view(Vs... bases);
```

Effects: Initialises bases_ with std::move(bases)....

```
constexpr iterator<false> begin() requires (!simple_view<Vs> || ...);
```

Effects: Equivalent to `return iterator<false>(tuple-transform(ranges::begin, bases_));`

```
constexpr iterator<true> begin() requires (range<Vs> && ...);
```

Effects: Equivalent to `return iterator<true>(tuple-transform(ranges::begin, bases_));`

```
constexpr iterator<false> end()  
requires ((!simple_view<Vs> || ...) && cartesian-product-is-common<Vs...>);
```

Effects: If every range in `Vs` models `random_access_range`, equivalent to `return begin() + size()`. Otherwise equivalent to:

```
iterator<false> it(tuple-transform(ranges::begin, bases_));  
std::get<0>(it.current_) = std::ranges::end(std::get<0>(bases_));  
return it;
```

```
constexpr iterator<true> end() const  
requires(cartesian-product-is-common<const Vs...>);
```

Effects: If every range in `Vs` models `random_access_range`, equivalent to `return begin() + size()`. Otherwise equivalent to:

```
iterator<true> it(tuple-transform(ranges::begin, bases_));  
std::get<0>(it.current_) = std::ranges::end(std::get<0>(bases_));  
return it;
```

```
constexpr default_sentinel_t end() const  
requires (!cartesian-product-is-common<const Vs...>) {
```

Effects: Equivalent to `return default_sentinel;`.

```
constexpr auto size() requires (sized_range<Vs> && ...);  
constexpr auto size() const requires (sized_range<const Vs> && ...);
```

Effects: Returns the product of the size of all ranges in `bases_`.

§ 4.1.3. Class template `cartesian_product_view::iterator` [ranges.cartesian.iterator]

```
namespace std::ranges {  
    template<forward_range... Vs>  
    requires (view<Vs> && ...)  
    template<bool Const>  
    class cartesian_product_view<Vs...>::iterator {  
        maybe-const<Const, cartesian_product_view>* parent_; //exposition only  
        tuple-or-pair<iterator_t<maybe-const<Const, Vs>>...> current_{}; //exposition only  
  
        template <size_t N = (sizeof...(Vs) - 1)>  
        void next(); //exposition only  
  
        template <size_t N = (sizeof...(Vs) - 1)>  
        void prev(); //exposition only  
  
    public:
```

```

using iterator_category = input_iterator_tag;
using iterator_concept = *see below*;
using value_type = tuple-or-pair<range_value_t<maybe-const<Const, Vs>>...>;
using difference_type = std::common_type_t<range_difference_t<maybe-const<Const, Vs>>...>;

iterator() = default;
constexpr explicit iterator(tuple-or-pair<iterator_t<maybe-const<Const, Vs>>...> current

constexpr iterator(iterator<!Const> i) requires Const && (std::convertible_to<
    std::ranges::iterator_t<Vs>,
    std::ranges::iterator_t<maybe-const<Const, Vs>>> && ...);

constexpr auto operator*() const;
constexpr iterator& operator++();
constexpr iterator operator++(int);

constexpr iterator& operator--()
requires (cartesian-product-is-bidirectional<maybe-const<Const, Vs>...>);
constexpr iterator operator--(int)
requires (cartesian-product-is-bidirectional<maybe-const<Const, Vs>...>);

constexpr iterator& operator+=(difference_type x)
requires (cartesian-product-is-random-access<maybe-const<Const, Vs>...>);
constexpr iterator& operator-=(difference_type x)
requires (cartesian-product-is-random-access<maybe-const<Const, Vs>...>);

constexpr reference operator[](difference_type n) const
requires (cartesian-product-is-random-access<maybe-const<Const, Vs>...>);

friend constexpr bool operator==(const iterator& x, const iterator& y)
requires (equality_comparable<iterator_t<maybe-const<Const, Vs>>> && ...);

friend constexpr bool operator==(const iterator& x, const std::default_sentinel_t&);

friend constexpr auto operator<(const iterator& x, const iterator& y)
requires (std::ranges::random_access_range<maybe-const<Const, Vs>> && ...);
friend constexpr auto operator>(const iterator& x, const iterator& y)
requires (std::ranges::random_access_range<maybe-const<Const, Vs>> && ...);
friend constexpr auto operator<=(const iterator& x, const iterator& y)
requires (std::ranges::random_access_range<maybe-const<Const, Vs>> && ...);
friend constexpr auto operator>=(const iterator& x, const iterator& y)
requires (std::ranges::random_access_range<maybe-const<Const, Vs>> && ...);

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
requires ((std::ranges::random_access_range<maybe-const<Const, Vs>> && ...) &&
    (std::ranges::three_way_comparable<maybe-const<Const, Vs>> && ...));

friend constexpr iterator operator+(const iterator& x, difference_type y)
requires (cartesian-product-is-random-access<maybe-const<Const, Vs>...>);
friend constexpr iterator operator+(difference_type x, const iterator& y)
requires (cartesian-product-is-random-access<maybe-const<Const, Vs>...>);
friend constexpr iterator operator-(const iterator& x, difference_type y)
requires (cartesian-product-is-random-access<maybe-const<Const, Vs>...>);
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
requires (sized_sentinel_for<iterator_t<maybe-const<Const, Vs>>, iterator_t<maybe-const
}
}

```

`iterator::iterator_concept` is defined as follows:

- If every type in `Vs` models `random_access_range`, then `iterator_concept` denotes `random_access_iterator_tag`.

- Otherwise, if every type in `Vs` models `bidirectional_range`, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- Otherwise, `iterator_concept` denotes `forward_iterator_tag`.

```
template <size_t N = (sizeof...(Vs) - 1)>
void next() //exposition only
```

Effects: Equivalent to:

```
auto& it = get<N>(current_);
++it;
if constexpr (N > 0) {
    if (it == ranges::end(get<N>(parent_->bases_)) {
        it = ranges::begin(get<N>(parent_->bases_));
        next<N - 1>();
    }
}
```

```
template <size_t N = (sizeof...(Vs) - 1)>
void prev() //exposition only
```

Effects: Equivalent to:

```
auto& it = std::get<N>(current_);
if (it == std::ranges::begin(std::get<N>(parent_->bases_)) {
    std::ranges::advance(it, std::ranges::end(std::get<N>(parent_->bases_)));
    if constexpr (N > 0) {
        prev<N - 1>();
    }
}
--it;
```

```
constexpr explicit iterator(tuple-or-pair<iterator_t<maybe-const<Const, Vs>>...> current);
```

Effects: Initializes `current_` with `std::move(current)`.

```
constexpr iterator(iterator<!Const> i)
    requires Const && (convertible_to<iterator_t<Vs>, iterator_t<maybe-const<Const, Vs>>> &&
```

Effects: Initializes `current_` with `std::move(i.current_)`.

```
constexpr auto operator*() const;
```

Effects: Equivalent to:

```
return tuple-transform([](auto& i) -> decltype(auto) { return *i; }, current_);
```

```
constexpr iterator& operator++();
```


Effects: Equivalent to:

```
next();  
return *this;
```

```
constexpr iterator operator++(int);
```

Effects: Equivalent to:

```
auto tmp = *this;  
++*this;  
return tmp;
```

```
constexpr iterator& operator--() requires (bidirectional_range<maybe-const<Const, Vs>> && .
```

Effects: Equivalent to:

```
prev();  
return *this;
```

```
constexpr iterator operator--(int) requires (bidirectional_range<maybe-const<Const, Vs>> &&
```

Effects: Equivalent to:

```
auto tmp = *this;  
--*this;  
return tmp;
```

```
constexpr iterator& operator+=(difference_type x) requires (random_access_range<maybe-const
```

Effects: Sets the position of the iterators in `current_`:

- If $x > 0$, as if `next` was called x times.
- Otherwise, if $x < 0$, as if `prev` was called x times.
- Otherwise, no effect.

```
constexpr iterator& operator-=(difference_type x) requires (random_access_range<maybe-const
```

Effects: Equivalent to:

```
*this += -x;  
return *this;
```

```
constexpr reference operator[](difference_type n) const requires (random_access_range<maybe
```

Effects: Equivalent to return `*((*this) + n);`.

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
requires (equality_comparable<iterator_t<maybe-const<Const, Vs>>> && ...);
```

Effects: Equivalent to return `x.current_ == y.current_;`.

```
friend constexpr auto operator<(const iterator& x, const iterator& y)
requires (random_access_range<maybe-const<Const, Vs>>> && ...);
```

Effects: Equivalent to return `x.current_ < y.current_;`.

```
friend constexpr auto operator>(const iterator& x, const iterator& y)
requires (random_access_range<maybe-const<Const, Vs>>> && ...);
```

Effects: Equivalent to return `y < x;`.

```
friend constexpr auto operator<=(const iterator& x, const iterator& y)
requires (random_access_range<maybe-const<Const, Vs>>> && ...);
```

Effects: Equivalent to return `!(y < x);`.

```
friend constexpr auto operator>=(const iterator& x, const iterator& y)
requires (random_access_range<maybe-const<Const, Vs>>> && ...);
```

Effects: Equivalent to return `!(x < y);`.

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
requires (random_access_range<maybe-const<Const, Vs>>>&&...) && (three_way_comparable<maybe-const<Const, Vs>>> && ...);
```

Effects: Equivalent to return `x.current_ <=> y.current_;`.

```
friend constexpr iterator operator+(const iterator& x, difference_type y)
requires (random_access_range<maybe-const<Const, Vs>>> && ...);
```

Effects: Equivalent to return `iterator{ x } += y;`.

```
friend constexpr iterator operator+(difference_type x, const iterator& y)
requires (random_access_range<maybe-const<Const, Vs>>> && ...);
```

Effects: Equivalent to return `y + x;`.

```
friend constexpr iterator operator-(const iterator& x, difference_type y)
requires (random_access_range<maybe-const<Const, Vs>>> && ...);
```

Effects: Equivalent to `return iterator{ x } -= y;`

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
requires (sized_sentinel_for<iterator_t<maybe-const<Const, Vs>>, iterator_t<maybe-const<Con
```

Effects: Let `scaled_distance(x,y,N)` be the distance between `(get<N>(x.current_) - get<N>(y.current_)) * get<N>(x.parent_ ->bases_)`. Returns the sum of `scaled_distance(x,y,N)` for every `N` in the interval `(0, sizeof...(Vs)]`.

§ 4.1.4. Class template `cartesian_product_view::sentinel` [ranges.cartesian.sentinel]

```
template <bool Const>
class sentinel {
    using parent = maybe-const<Const, cartesian_product_view>; //exposition only
    using first_base = decltype(get<0>(std::declval<parent>().bases_)); //exposition only
    sentinel_t<first_base> end_; //exposition only

public:
    sentinel() = default;
    sentinel(std::ranges::sentinel_t<first_base> end);

    friend constexpr bool operator==(sentinel const& s, iterator_t<parent> const& it);

    constexpr sentinel(sentinel<!Const> other)
    requires Const &&
        (std::convertible_to<std::ranges::sentinel_t<first_base>, std::ranges::sentinel_t<
```

```
sentinel(std::ranges::sentinel_t<first_base> end);
```

Effects: Initialises `end_` with `std::move(end)`.

```
constexpr sentinel(sentinel<!Const> other)
requires Const &&
    (std::convertible_to<std::ranges::sentinel_t<first_base>, std::ranges::sentinel_t<const
: end_(other.end_) {}
```

Effects: Initialises `end_` with `std::move(other.end_)`.

```
friend constexpr bool operator==(sentinel const& s, iterator_t<parent> const& it);
```

Effects: Equivalent to `return get<0>(it.current_) == s.end_;`

§ 5. Acknowledgements

Thank you to Christopher Di Bella, Corentin Jabot, and Barry Revzin for feedback and guidance.